Automating Quality Assurance for Mojaloop Deployments

From framework artefacts to machine-readable assurance



Presented by Moses Kipchirchir, KPMG East Africa

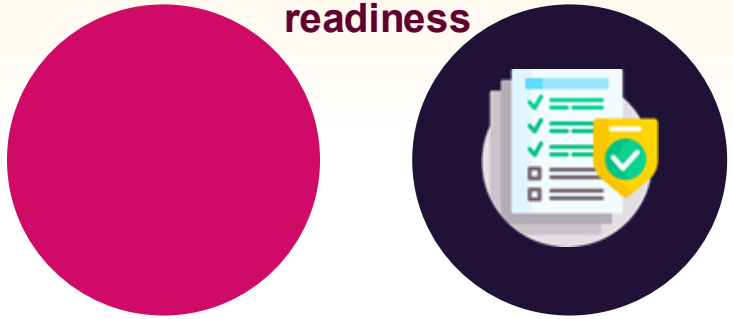
Importance of the QA Framework

Mojaloop deployments are part of critical payment infrastructure. Assurance must become as repeatable and scalable as the platform itself.

Confidence in deployment readiness

Adopters

Faster readiness checks before go-live



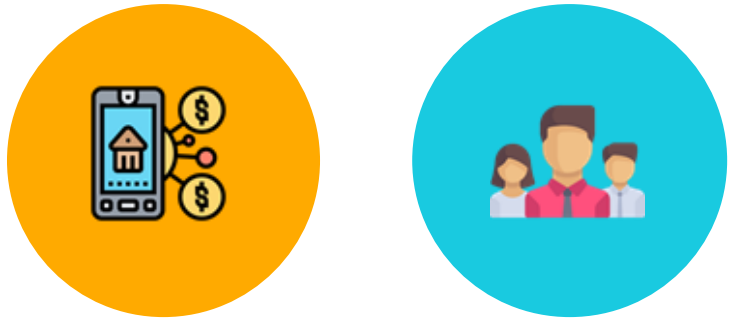
Adopters

Faster readiness checks before go-live

DFSPs

Greater trust in integration quality

.



End users

Reliable and resilient payment flows

The opportunity: reduce manual friction while preserving the auditability and professional judgement that assurance requires.

Recap: what the QA Framework already provides

The current framework is already useful as it defines what good looks like and what evidence an implementation should be able to produce.



Framework overview

Purpose, scope, applicability and six QA pillars

- Configuration
- Security
- Functionality
- Performance
- Interoperability
- Overlay services



Controls catalogue

- Control Areas & Sub-areas
- Control description
- Objectives
- Procedures
- Evidence to be collected and processed



Self-assessment checklist

Yes / No assessment, evidence reference and comments

The six QA pillars remain the backbone

Automation starts by preserving the community-approved assurance logic, then making it structured enough for tools to read.



Configuration

- Correct setup
- Environment readiness:
  - Environment Variable Configuration
  - Database Configuration



Security

- Identity and Access Management (IAM)
- Role-Based Access Control (RBAC)
- Secret Management
- Secure Communication Protocols
- Vulnerability Management and Penetration Testing



Functionality

- Core Transaction Processing (Transaction Flow)
- Error Handling and Reversal
- Use Case Validation



Performance

- Performance Benchmarking(TPS, Latency)
- System Availability and Resilience
- Scalability and Resilience Testing



Interoperability

- FSPIOP API Conformance
- ISO 20022 Messaging Compliance
- Cross-Scheme Interoperability
- Participant Onboarding Process



Overlay services

- Safe extension without weakening the core platform

The limitation: the framework is still document-bound

Word and Excel are good for human review. They are weak interfaces for software automation, evidence ingestion and AI-assisted analysis.

Current status today


Narrative overview


Controls catalogue


Self-assessment


Manual



AUTOMATION CEILING

Different adopters can interpret the same control differently.

Evidence collection depends on manual chasing and judgement.

Controls are not consumable by parsers, or CI/CD checks.

Software parsers and AI have no reliable structure for linking controls, evidence and exceptions.

The next step is not to abandon the framework , it is to make it machine-readable.

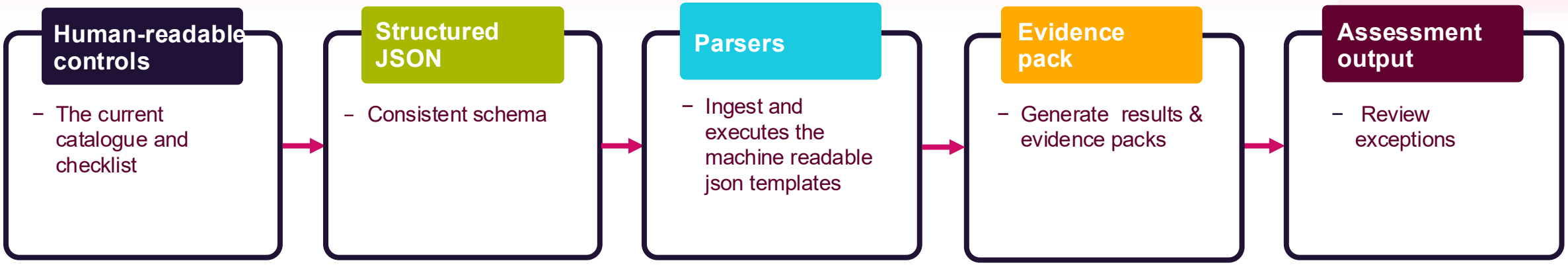
THE SHIFT

From checklist to executable knowledge

Same assurance logic. More structured format. Greater automation potential.

What machine-readable assurance changes

Structured controls allow QA logic to be reused across parsers, reviewer workflows and later AI assistance.



A real machine-readable control

This is not illustrative , it is rule ENV-VAR-001 from spec v1.3.0 of the Configuration pillar template compared with the existing excel based templates

Excel

2. Validating **environment variables** for core Mojaloop services e.g., the Central Ledger and calllback handlers.
3. Verify the **configuration of Kafka topic mappings**, such as KAFKA.EVENT_TYPE_ACTION_TOPIC_MAP. Also, validating **batch processing parameters** such as KAFKA.CONSUMER.TRANSFER.POSITION_BATCH.config.options.batchSize.

```
{
  "rule_id": "ENV-VAR-001",
  "rule_name": "Required Core Environment
     Variables Presence Validation",
  "type": "presence_check",
  "severity": "critical",
  "execution": { "mode": "automated" },
  "automatable": true,

  "logic": {
    "operator": "all_present",
    "operands": [
      "CLEDG_DATABASE_URI", "KAFKA_HOST",
      "KAFKA_PORT", "ALS_DATABASE_URI",
      "ML_API_ADAPTER_PORT"
    ]
  },

  "evidence": {
    "artifacts": [".env", "docker-compose.yml",
                  "values.yaml"],
    "setup_commands": ["cat .env"]
  }
}
```

Pilot pillar: Configuration Pillar

Configuration was chosen because the primary foundation for any Mojaloop deployment and its checks are evidence-rich, repeatable and directly tied to deployment readiness.

Environment variables

Validate required values across Dev/QA/Prod

Database readiness

Check connectivity, schema migration and persistent storage



Configuration pilot

Runtime configuration

Compare deployed values against the approved template

Kafka and Helm / IaC

Confirm mappings, values.yaml overrides and deployment descriptors

Starting focused allows the community to test the schema, parser and evidence model before scaling.

Progress and near-term roadmap

The workstream is moving through a focused pilot: make Configuration machine-readable with a software parser, then use the learnings to scale.
Near-term success means proving the schema and the software parser on one pillar , before scaling to the rest of the QA Framework, and long before any AI facilitation.



April 2026 – Kickoff and Requirements
Align on template format and pilot pillar

June 2026 – Json Schema Definition
Define the schema structure
Translate the Configuration pillar to the defined schema

July 2026 – Parser Development:
Build a parser capable of ingesting the JSON-schema QA pillar and extracting its contents

August 2026 - Testing
Perform tests in a demo environments.

Sept 2026 – Finalize the Configuration pillar
Finalize the configuration pillar and work out a plan to extend to the next pillars

Scaling beyond the Configuration pilot

The pilot should produce a reusable schema pattern that applies across the remaining pillars without losing domain-specific detail.



PILOT

Configuration

- Correct setup
- Environment readiness:
  - Environment Variable Configuration
  - Database Configuration



NEXT

Security

- Identity and Access Management (IAM)
- Role-Based Access Control (RBAC)
- Secret Management
- Secure Communication Protocols
- Vulnerability Management and Penetration Testing



NEXT

Functionality

- Core Transaction Processing (Transaction Flow)
- Error Handling and Reversal
- Use Case Validation



NEXT

Performance

- Performance Benchmarking(TPS, Latency)
- System Availability and Resilience
- Scalability and Resilience Testing



NEXT

Interoperability

- FSPIOP API Conformance
- ISO 20022 Messaging Compliance
- Cross-Scheme Interoperability
- Participant Onboarding Process



NEXT

Overlay services

- Safe extension without weakening the core platform

Critical scaling question: *which controls should become automated checks, which should become evidence pre-checks, and which must remain reviewer-led?*

What this means for adopters

The practical value is a shift from a one-off QA pack to a living assurance workflow that supports implementation and ongoing operation.

BEFORE

- Document-based QA
- Manual self-assessment
- Evidence references maintained by each team
- Limited comparability across deployments

AFTER



Manual based evaluation



Machine-readable assurance workflow

Thank you